

Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nist.gov/aands>, Chapter 17.[13]

Mathews, J., and Walker, R.L. 1970, *Mathematical Methods of Physics*, 2nd ed. (Reading, MA: W.A. Benjamin/Addison-Wesley), pp. 78–79.

6.13 Hypergeometric Functions

As was discussed in §5.14, a fast, general routine for the the complex hypergeometric function ${}_2F_1(a, b, c; z)$ is difficult or impossible. The function is defined as the analytic continuation of the hypergeometric series

$$\begin{aligned} {}_2F_1(a, b, c; z) = & 1 + \frac{ab}{c} \frac{z}{1!} + \frac{a(a+1)b(b+1)}{c(c+1)} \frac{z^2}{2!} + \cdots \\ & + \frac{a(a+1) \dots (a+j-1)b(b+1) \dots (b+j-1)}{c(c+1) \dots (c+j-1)} \frac{z^j}{j!} + \cdots \end{aligned} \quad (6.13.1)$$

This series converges only within the unit circle $|z| < 1$ (see [1]), but one's interest in the function is not confined to this region.

Section 5.14 discussed the method of evaluating this function by direct path integration in the complex plane. We here merely list the routines that result.

Implementation of the function `hypgeo` is straightforward and is described by comments in the program. The machinery associated with Chapter 17's routine for integrating differential equations, `Odeint`, is only minimally intrusive and need not even be completely understood: Use of `Odeint` requires one function call to the constructor, with a prescribed format for the derivative routine `Hypderiv`, followed by a call to the `integrate` method.

The function `hypgeo` will fail, of course, for values of z too close to the singularity at 1. (If you need to approach this singularity, or the one at ∞ , use the “linear transformation formulas” in §15.3 of [1].) Away from $z = 1$, and for moderate values of a, b, c , it is often remarkable how few steps are required to integrate the equations. A half-dozen is typical.

```
hypgeo.h  Complex hypgeo(const Complex &a, const Complex &b, const Complex &c,
               const Complex &z)
Complex hypergeometric function  ${}_2F_1$  for complex  $a, b, c$ , and  $z$ , by direct integration of the
hypergeometric equation in the complex plane. The branch cut is taken to lie along the real
axis,  $\text{Re } z > 1$ .
{
    const Doub atol=1.0e-14, rtol=1.0e-14;          Accuracy parameters.
    Complex ans, dz, z0, y[2];
    VecDoub yy(4);
    if (norm(z) <= 0.25) {                          Use series...
        hypser(a, b, c, z, ans, y[1]);
        return ans;
    }
    ...or pick a starting point for the path integration.
    else if (real(z) < 0.0) z0=Complex(-0.5, 0.0);
    else if (real(z) <= 1.0) z0=Complex(0.5, 0.0);
```

```

else z0=Complex(0.0,imag(z) >= 0.0 ? 0.5 : -0.5);
dz=z-z0;
hypser(a,b,c,z0,y[0],y[1]);           Get starting function and derivative.
yy[0]=real(y[0]);
yy[1]=imag(y[0]);
yy[2]=real(y[1]);
yy[3]=imag(y[1]);
Hypderiv d(a,b,c,z0,dz);              Set up the functor for the derivatives.
Output out;                           Suppress output in Odeint.
Odeint<StepperBS<Hypderiv> > ode(yy,0.0,1.0,atol,rtol,0.1,0.0,out,d);
The arguments to Odeint are the vector of independent variables, the starting and ending
values of the dependent variable, the accuracy parameters, an initial guess for the stepsize,
a minimum stepsize, and the names of the output object and the derivative object. The
integration is performed by the Bulirsch-Stoer stepping routine.
ode.integrate();
y[0]=Complex(yy[0],yy[1]);
return y[0];
}

```

```

void hypser(const Complex &a, const Complex &b, const Complex &c,           hypgeo.h
            const Complex &z, Complex &series, Complex &deriv)

```

Returns the hypergeometric series ${}_2F_1$ and its derivative, iterating to machine accuracy. For $|z| \leq 1/2$ convergence is quite rapid.

```

{
    deriv=0.0;
    Complex fac=1.0;
    Complex temp=fac;
    Complex aa=a;
    Complex bb=b;
    Complex cc=c;
    for (Int n=1;n<=1000;n++) {
        fac *= ((aa*bb)/cc);
        deriv += fac;
        fac *= ((1.0/n)*z);
        series=temp+fac;
        if (series == temp) return;
        temp=series;
        aa += 1.0;
        bb += 1.0;
        cc += 1.0;
    }
    throw("convergence failure in hypser");
}

```

```

struct Hypderiv {                                                         hypgeo.h

```

Functor to compute derivatives for the hypergeometric equation; see text equation (5.14.4).

```

    Complex a,b,c,z0,dz;
    Hypderiv(const Complex &aa, const Complex &bb,
              const Complex &cc, const Complex &z00,
              const Complex &dzz) : a(aa),b(bb),c(cc),z0(z00),dz(dzz) {}
    void operator() (const Doub s, VecDoub_I &yy, VecDoub_O &dyyds) {
        Complex z,y[2],dyds[2];
        y[0]=Complex(yy[0],yy[1]);
        y[1]=Complex(yy[2],yy[3]);
        z=z0+s*dz;
        dyds[0]=y[1]*dz;
        dyds[1]=(a*b*y[0]-(c-(a+b+1.0)*z)*y[1])*dz/(z*(1.0-z));
        dyyds[0]=real(dyds[0]);
        dyyds[1]=imag(dyds[0]);
        dyyds[2]=real(dyds[1]);
        dyyds[3]=imag(dyds[1]);
    }
};

```